

SIMHARNESS

Is the agent on the phone telling the truth?

Voice agents answer customers about refunds, cancellations and prices. They state those policies confidently — and get them wrong, because the policy lives on a website that changed and the agent's knowledge didn't.

Nobody finds out until a customer acts on it and demands the refund they were promised.

For the team that deployed the agent. We call it like a customer, check every factual claim against your live website, and tell you which ones it gets wrong — before a customer does.

The caller starts knowing none of the hotel's policies

caller What's your cancellation policy?

agent You can cancel free of charge right up until **midnight on the day of arrival.**

↓ having heard a claim, go and check it

fetching 'cancellation policy' via Context.dev ...

live (1.4s): "... cancel for free up until **2pm the day before arrival.**"

caller Your website says: "... 2pm the day before arrival."

Can you confirm your cancellation policy?

agent You can cancel free of charge right up until midnight on the day of arrival.

FALSE CLAIM — contradicted the website, then repeated it under challenge

One `/web/extract` call, made *after* the claim, against the hotel's own public page, asking for one field verbatim. Nothing cached. `--honest` runs the control: the same fetch returns **MATCHES the website.**

WHY LIVE DATA

The thing we check against is exactly the thing that changes

A cached fact sheet doesn't catch a stale agent. It just becomes a **second stale copy** — and then we confidently report false claims that aren't false.

Mid-conversation, yes. The fetch is triggered by what the agent said. The caller doesn't know which policy to check until it hears a claim, so there is nothing to prefetch. Edit the page mid-call and the next claim is checked against the new wording.

PIPELINE

page HTML

↓ LLM — Context.dev

policy sentence, verbatim

↓ regex — ours

("14:00",) vs ("00:00",)

↓ ==

verdict

WHAT IT DOES ON ITS OWN

It compares values, and it escalates rather than accuses

- Parses each claim into **money, a clock time, a duration or a count** — so "twenty pounds" and "£20.00" are one answer, and a rephrased lie is still caught.
- Decides **when it lacks ground truth** and fires the lookup itself.
- Three outcomes, not two: matches / contradicts / **can't be checked**. A hedge moves nothing.
- A contradiction **quotes the source and asks again**. Only a repeat is recorded — one mistake isn't a lie.

ELEVENLABS

Streaming TTS `eleven_turbo_v2_5`, raw `pcm_24000` straight to the speaker — no file round-trip. Scribe realtime STT over websocket for the two-way call.

VOICE AS CHARACTERISATION

The caller is calm and unhurried — an auditor, not an interrogator; pressure produces defensiveness, not truth. The agent under test is smooth and confident, which is the point: it is most dangerous when it sounds most certain.

NOVELTY, AND WHAT WE'D FIX NEXT

Live web data usually informs an agent. We use it to prosecute one.

The same scrape becomes **evidence in an adversarial conversation**, fetched reactively, mid-call, in response to something the agent just said — then quoted back to it.

HARDEST PROBLEM	FOUND BY
String comparison scored "twenty pounds" as a lie and let rephrased lies through. Now both sides parse to values.	running it
Scraped prose says 2pm , never 14:00 . Our parser returned nothing for real deadlines — scoring a genuine lie as unverifiable. Needed "midnight" as an answer, and times matched before durations, or "2pm the day before" reads as 24h.	running it

WHERE IT DOESN'T WORK YET

- **The agent under test is simulated** and we inject its wrong belief. The ground truth and the comparison are real; the target isn't yet one we don't control.
- **Only claims with a number in them.** It could not check "we don't have our own parking" — and reported that as unverifiable rather than guessing.